

## **MedAssist AI**

### **Milestone 4 – Final Implementation, Deployment and Documentation**

---

#### **1. Introduction**

MedAssist AI is an AI-powered healthcare assistance web application designed to help users understand their health symptoms through an interactive, guided workflow. Users can register and log in securely, select symptoms across a range of medical categories, and submit them for AI-based analysis. The system returns a predicted condition along with confidence and risk information, relevant recommendations, and suggested tests. Users can also maintain a history of their medical consultations as reports and download these reports as PDF documents for personal record-keeping or for sharing with a healthcare provider.

#### **Problem Addressed**

Many individuals experience symptoms and are uncertain about their possible causes, severity, or the appropriate next steps to take. MedAssist AI addresses this gap by providing a structured, easy-to-use platform where users can describe their symptoms and receive AI-generated informational guidance, along with a persistent record of their consultations.

#### **Purpose of Milestone 4**

Milestone 4 represents the final implementation phase of the project. It focuses on completing the core application features, integrating the AI prediction pipeline with the backend and database, deploying the frontend and backend to production, and validating the end-to-end functionality of the system.

#### **Summary of Work Completed in Milestone 4**

- Finalized the React/Vite frontend, including authentication flows, the symptom checker, prediction results, and report management screens.
- Completed the FastAPI backend, including JWT-based authentication, symptom category management, AI-based prediction handling, and report generation/retrieval endpoints.
- Integrated the Groq API for AI-based symptom analysis using the openai/gpt-oss-120b model.
- Connected the backend to MongoDB for persistent storage of users, symptoms, and consultation records.

- Deployed both the frontend and backend to Render, and configured CORS and environment-based secrets for production use.
  - Resolved a series of implementation and deployment issues encountered during integration and testing (documented in Section 11).
- 

## 2. Project Objectives

- Provide users with a secure account system for accessing personalized health features.
  - Allow users to browse and select symptoms organized by medical category.
  - Submit selected symptoms to an AI model for informational analysis and receive a predicted condition, confidence level, risk level, and reasoning.
  - Present users with actionable recommendations, suggested tests, and precautions based on the AI's response.
  - Maintain a persistent, user-specific history of consultations as medical reports.
  - Allow users to download individual reports as PDF documents.
  - Deploy the complete application (frontend and backend) to a publicly accessible production environment.
  - Ensure the system is clearly positioned as an informational assistance tool and not a diagnostic or medical device.
- 

## 3. System Overview

The application follows a client-server architecture. The React frontend communicates with the FastAPI backend over HTTPS using REST API calls. The backend authenticates requests using JWT tokens, retrieves and processes symptom data, forwards prediction requests to the Groq AI API, and persists data in MongoDB.

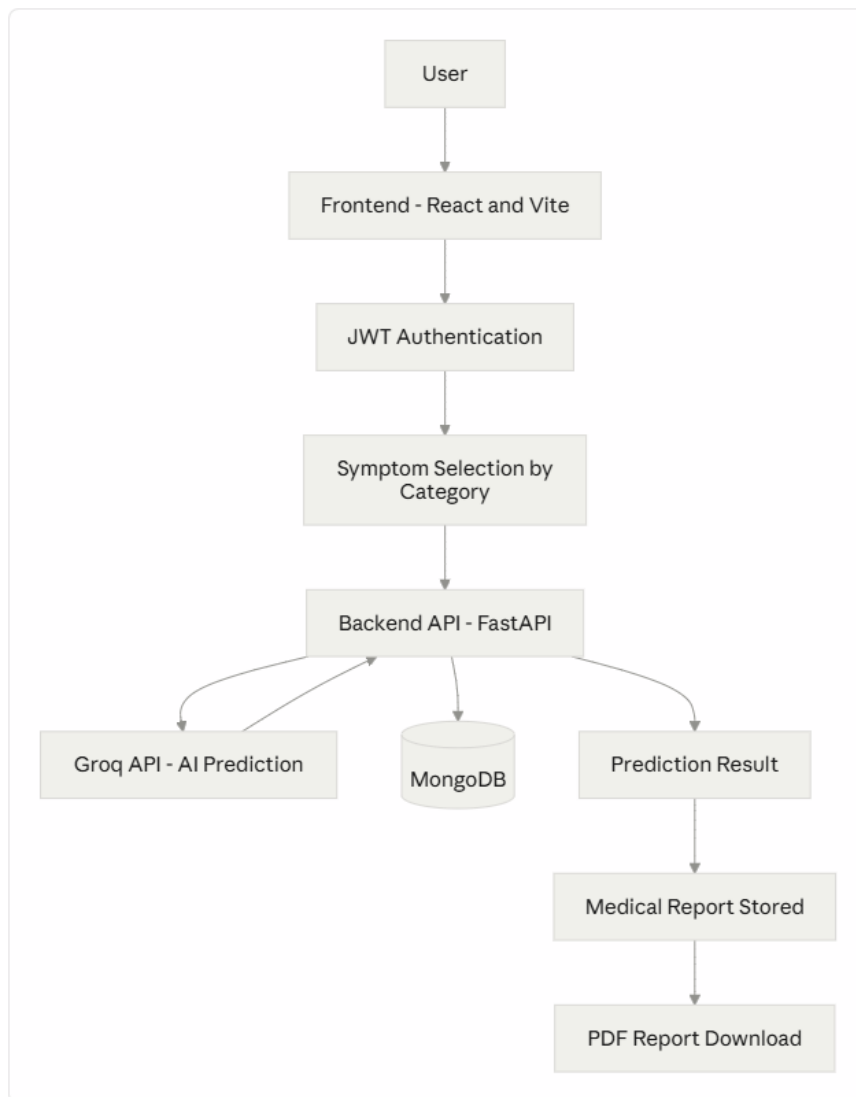
### High-Level Workflow

User

- Frontend (React/Vite)
- Authentication (JWT)
- Symptom Selection (by category)
- Backend API (FastAPI)

- AI Prediction (Groq API – openai/gpt-oss-120b)
- Database (MongoDB)
- Prediction Result (confidence, risk, recommendations, tests)
- Medical Report (saved consultation record)
- PDF Download

### Architecture Diagram



---

## 4. Technology Stack

| Technology                           | Purpose                                                       |
|--------------------------------------|---------------------------------------------------------------|
| React                                | Frontend UI library used to build the application's interface |
| Vite                                 | Frontend build tool and development server                    |
| JavaScript                           | Primary language for frontend logic                           |
| CSS                                  | Styling of frontend components and pages                      |
| React Router                         | Client-side routing/navigation between application pages      |
| Chart.js / frontend chart components | Visualization of data where applicable in the UI              |
| Python                               | Primary language for backend development                      |
| FastAPI                              | Backend web framework used to build REST API endpoints        |
| Uvicorn                              | ASGI server used to run the FastAPI application               |
| JWT (JSON Web Tokens)                | Authentication and session management                         |
| Groq API                             | AI inference provider used for symptom analysis               |
| MongoDB                              | Persistent NoSQL database for storing application data        |
| Motor / PyMongo                      | Python drivers used to interact with MongoDB                  |
| Render                               | Cloud platform used to deploy both frontend and backend       |

---

## 5. Application Features

### 5.1 User Authentication

MedAssist AI implements a registration and login system so that each user's data — symptom selections, prediction history, and reports — remains private and tied to their

account. Upon successful login, the backend issues a JWT token, which the frontend attaches to subsequent API requests to authenticate and authorize access to protected endpoints. This ensures that only authenticated users can access personal data such as consultation history and reports.

## **5.2 Dashboard**

[To be filled — describe dashboard contents based on confirmed screenshots/source, e.g., health overview, quick actions, recent consultations]

## **5.3 Symptom Checker**

The symptom checker allows users to search for and select symptoms relevant to what they are experiencing. Symptoms are organized into medical categories to make navigation easier, particularly when the number of available symptoms is large. The backend currently supports the following medical categories:

- General
- Cardiology
- Pulmonology
- Dermatology
- Gastroenterology
- ENT
- Pain & Discomfort
- Mental Health
- Gynecology
- Musculoskeletal
- Ophthalmology
- Urology

Users can filter symptoms by category, search for specific symptoms, and build a list of selected symptoms. Note that categories are not guaranteed to contain an equal number of symptoms; the distribution depends on the underlying symptom dataset. Once the user is satisfied with their selection, they proceed to submit the symptoms for AI-based analysis.

## **5.4 AI-Based Disease Prediction**

Once a user submits their selected symptoms, the frontend sends the data to the FastAPI backend, which forwards a structured prompt to the Groq AI model for

processing. The AI-generated response is parsed by the backend and includes the following fields:

- Prediction
- Confidence
- Risk
- Reason / Explanation
- Recommendations
- Tests
- Precautions
- When to see a doctor
- Selected symptoms

**Important:** MedAssist AI is an informational AI assistance system. It is **not** a medical diagnostic tool and does not replace professional medical advice, diagnosis, or treatment. Users experiencing a medical emergency or serious symptoms are advised to seek professional medical care.

### 5.5 Prediction Result

After analysis, the application displays a prediction result page presenting the predicted condition, the AI's confidence level, an associated risk level, an explanation of the reasoning behind the prediction, recommended next steps, and suggested tests. This gives users a clear, structured summary of the AI's output rather than a single unexplained label.

### 5.6 Medical Reports

Each completed prediction is stored as a user-specific consultation/report record. This allows users to review their past symptom submissions and corresponding AI analyses over time, effectively maintaining a personal health consultation history within the application.

### 5.7 PDF Report Generation

Users can download an AI-generated medical report as a PDF document for a given consultation. This allows the information to be saved locally or shared outside the application, such as with a healthcare provider.

### 5.8 User Profile / Other Implemented Features

[To be filled — include only features confirmed from project files/screenshots, e.g., profile editing, settings]

---

## 6. Backend API Architecture

The backend is built using FastAPI, a modern Python web framework, and served using Uvicorn. It exposes a set of REST API endpoints that handle authentication, symptom data retrieval, AI-based prediction, and report management. Confirmed endpoints include:

| Method | Endpoint                          | Purpose                                                    |
|--------|-----------------------------------|------------------------------------------------------------|
| GET    | /                                 | Base/health check endpoint                                 |
| POST   | /api/predictions/analyze          | Submits selected symptoms for AI-based analysis            |
| GET    | /api/predictions/{prediction_id}  | Retrieves a specific prediction result                     |
| GET    | /api/symptoms/categories          | Retrieves the list of available medical symptom categories |
| GET    | /api/symptoms                     | Retrieves available symptoms                               |
| GET    | /api/reports                      | Retrieves the list of a user's saved reports               |
| GET    | /api/reports/{report_id}/download | Downloads a specific report as a PDF                       |

**Authentication:** Protected endpoints require a valid JWT token, issued at login, to be included with each request. The backend validates this token before processing the request and associates the request with the corresponding authenticated user.

Full interactive API documentation (Swagger UI) is available at: <https://medassistai-unpq.onrender.com/docs>

Additional endpoints, request/response schemas, and parameter details beyond those listed above should be confirmed directly from the Swagger documentation or backend source files.

---

## 7. Database Design

MedAssist AI uses MongoDB as its persistent data store, accessed through Motor/PyMongo. The following collections are confirmed as part of the project:

| Collection       | Purpose                                                                                                   |
|------------------|-----------------------------------------------------------------------------------------------------------|
| users            | Stores user account information used for authentication                                                   |
| profiles         | Stores user profile-related information                                                                   |
| symptoms         | Stores the catalog of symptoms available for selection, organized by category                             |
| disease_profiles | Stores reference data associated with predicted conditions                                                |
| consultations    | Stores individual user consultation/report records, including selected symptoms and AI prediction results |

Specific field-level schemas for each collection should be confirmed from the backend source files before being added to the final documentation, as these were not provided.

---

## 8. AI Integration

AI-based symptom analysis is powered by the Groq API, using the model **openai/gpt-oss-120b**. When a user submits their selected symptoms, the backend constructs a prompt containing the relevant symptom data and sends it to the Groq API for inference.

The AI model returns a structured response, which the backend parses as JSON. This response includes the prediction, confidence, risk, reasoning, recommendations, suggested tests, precautions, and guidance on when to see a doctor. After receiving the AI's response, the backend appends the user's originally selected symptoms to the result before returning it to the frontend, ensuring the final prediction record reflects exactly which symptoms were analyzed.

No measured accuracy statistic for the AI model is available at this time; none is claimed in this documentation.

---

## 9. Deployment

MedAssist AI is deployed using Render for both the frontend and backend services.

- **Frontend:** Deployed as a static/web service built with Vite, hosted at <https://medassist-ai-frontend.onrender.com>.
- **Backend:** Deployed as a Python/FastAPI service, served via Uvicorn, hosted at <https://medassistai-unpq.onrender.com>.
- **Database:** MongoDB is used for persistent storage, connected to the backend service.



The frontend communicates with the backend exclusively over HTTPS, using REST API calls to the endpoints described in Section 6. To allow the deployed frontend to make requests to the backend, Cross-Origin Resource Sharing (CORS) was configured on the backend to explicitly allow the frontend's deployed origin.

---

## 10. Testing and Validation

The following table outlines the key test cases relevant to the application's functionality. Status values marked as "[To be verified]" should be confirmed and updated before final submission.

| Test Case                    | Expected Result                                                                       | Status |
|------------------------------|---------------------------------------------------------------------------------------|--------|
| Frontend accessibility       | Frontend loads successfully at the deployed URL                                       | Passed |
| Backend health endpoint      | Base endpoint (GET /) returns a successful response                                   | Passed |
| User authentication          | User can register and log in; JWT token is issued and accepted on protected routes    | Passed |
| Symptom category retrieval   | GET /api/symptoms/categories returns the list of available categories                 | Passed |
| Symptom retrieval            | GET /api/symptoms returns available symptoms                                          | Passed |
| Symptom analysis submission  | POST /api/predictions/analyze accepts selected symptoms and returns a prediction      | Passed |
| AI prediction generation     | Response includes prediction, confidence, risk, reasoning, recommendations, and tests | Passed |
| Report creation              | Completed prediction is saved as a consultation/report record                         | Passed |
| Report retrieval             | GET /api/reports returns the user's saved reports                                     | Passed |
| PDF report download          | GET /api/reports/{report_id}/download returns a downloadable PDF                      | Passed |
| Frontend/backend integration | Frontend correctly sends requests to and renders responses from the deployed backend  | Passed |

---

## 11. Error Handling and Issues Resolved

During development and deployment, several implementation issues were identified and resolved:

- **CORS configuration between Render frontend and backend:** Initial cross-origin requests from the deployed frontend to the backend were blocked. This was resolved by explicitly configuring the backend's CORS middleware to allow the deployed frontend origin.
- **Outdated/unavailable Groq model:** An earlier Groq model used for AI prediction became outdated or unavailable, causing prediction requests to fail. This was resolved by switching to the currently available model, openai/gpt-oss-120b.
- **Authentication/JWT handling:** Issues related to token validation and protected route access were identified during integration testing and addressed to ensure consistent authentication behavior across the frontend and backend.
- **MongoDB ObjectId serialization problem in reports:** MongoDB's ObjectId type is not natively JSON-serializable, which caused errors when returning report data through the API. This was resolved by converting ObjectId values to strings before returning them in API responses.
- **Report download database lookup issue:** An issue was identified in the lookup logic used when retrieving a specific report for PDF download, which was corrected to ensure the correct report record is retrieved for the requesting user.
- **Deployment/API connectivity issues:** Initial connectivity issues between the deployed frontend and backend (e.g., incorrect API base URLs) were identified and resolved to ensure reliable communication in the production environment.

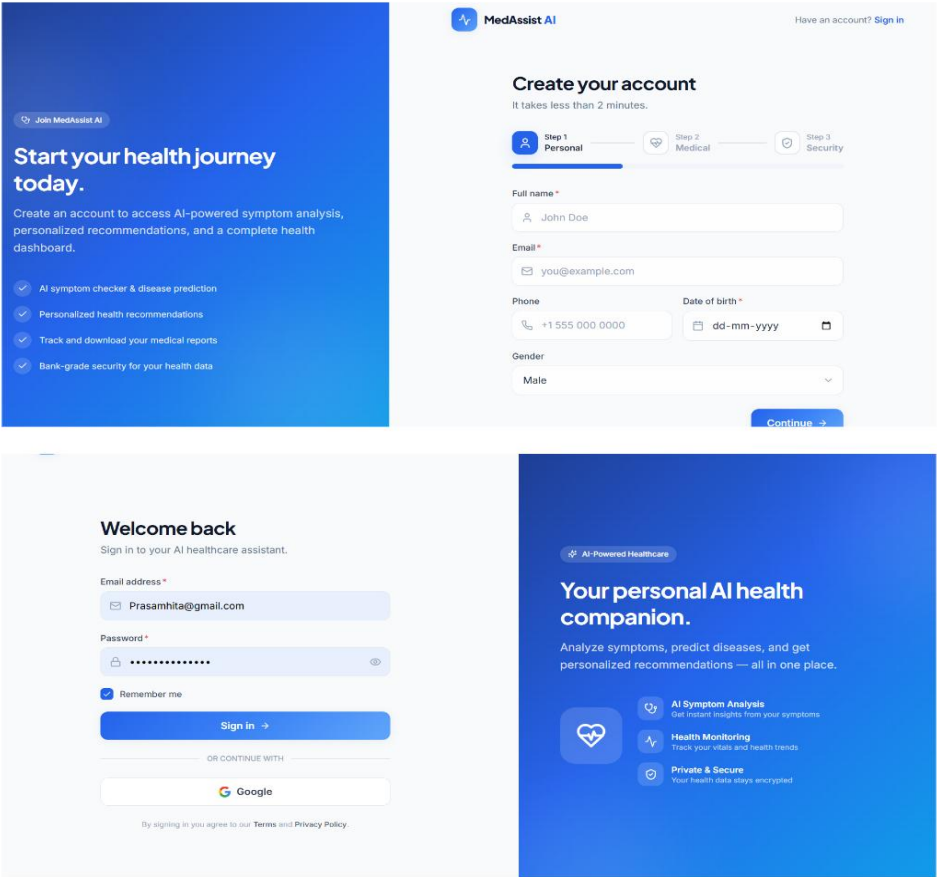
These issues were identified and resolved as part of the standard development and deployment process, and the application is functional in its current deployed state.

---

## 12. Screenshots

*Screenshots to be inserted below in the recommended order. Replace placeholder paths with the actual screenshot files once provided.*

1. Figure 1. MedAssist AI Login/Register page, showing the user authentication interface.



The image displays the MedAssist AI user authentication interface, which is split into two main sections: a registration page on the left and a login page on the right.

**Registration Page (Left):**

- Header:** MedAssist AI logo and a link "Have an account? Sign in".
- Section:** "Create your account" with a subtext "It takes less than 2 minutes."
- Progress:** A three-step progress bar: Step 1 Personal (active), Step 2 Medical, and Step 3 Security.
- Form Fields:**
  - Full name \* (Text input: John Doe)
  - Email \* (Text input: you@example.com)
  - Phone (Text input: +1 555 000 0000)
  - Date of birth \* (Text input: dd-mm-yyyy)
  - Gender (Dropdown menu: Male)
- Buttons:** "Continue" button at the bottom right.

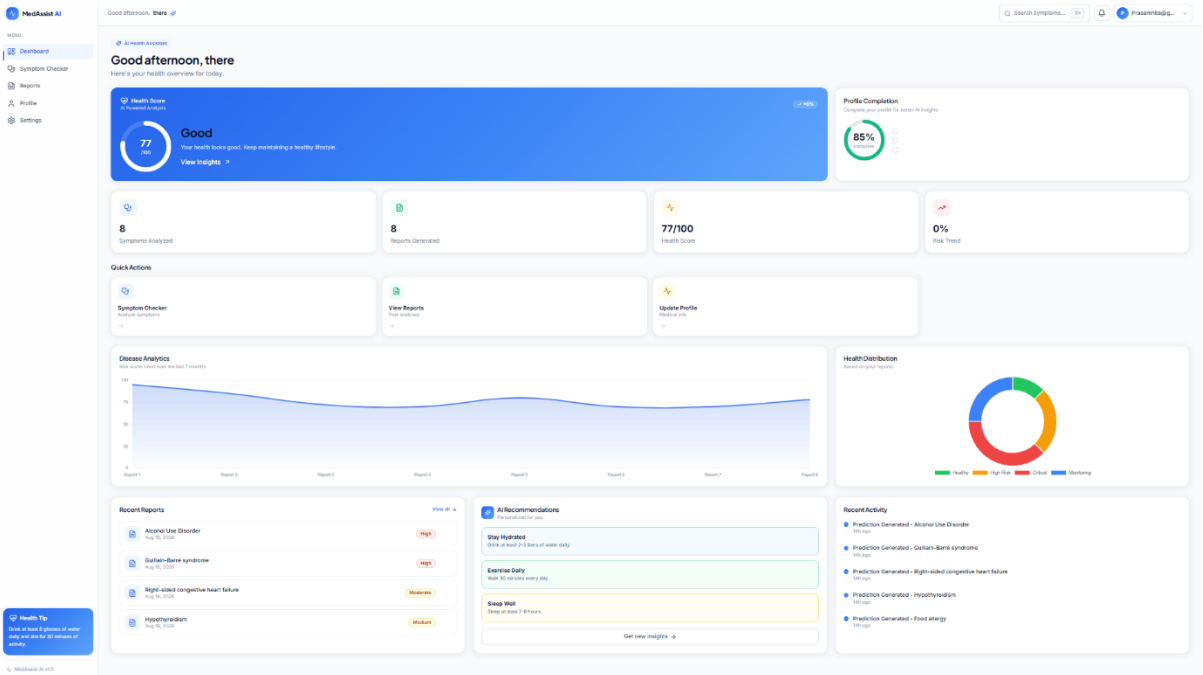
**Login Page (Right):**

- Section:** "Welcome back" with a subtext "Sign in to your AI healthcare assistant."
- Form Fields:**
  - Email address \* (Text input: Prasamhita@gmail.com)
  - Password \* (Text input: masked with dots)
  - ☒ Remember me
- Buttons:** "Sign in" button, "OR CONTINUE WITH" section with a "Google" button.
- Footer:** "By signing in you agree to our Terms and Privacy Policy."

**Right Panel (Blue Gradient):**

- Section:** "Your personal AI health companion."
- Text:** "Analyze symptoms, predict diseases, and get personalized recommendations — all in one place."
- Features:**
  - AI Symptom Analysis: Get instant insights from your symptoms.
  - Health Monitoring: Track your vitals and health trends.
  - Private & Secure: Your health data stays encrypted.

2. Figure 2. MedAssist AI Dashboard showing the user's health overview and quick actions.



3. Figure 3. Symptom Checker interface, showing symptom search and selection.

MedAssist AI

Good afternoon, there

Search symptoms... 35x

Prasamhita@g...

MENU

- Dashboard
- Symptom Checker
- Reports
- Profile
- Settings

Let's analyze your symptoms

Tell us how you're feeling and our AI will assess possible conditions.

Step 1 of 3 33%

Search symptoms (e.g. headache, fever)...

General Cardiology Pulmonology Dermatology Gastroenterology ENT

Pain & Discomfort Mental Health Gynecology Musculoskeletal Ophthalmology

Urology

abdominal distention abnormal involuntary movements absence of menstruation

abusing alcohol allergic reaction ankle stiffness or tightness

ankle swelling ankle weakness apnea

arm cramps or spasms arm lump or mass arm stiffness or tightness

arm swelling arm weakness bedwetting

bleeding gums bleeding in mouth bleeding or discharge from nipple

Health Tip

Drink at least 8 glasses of water daily and aim for 30 minutes of activity.

MedAssist AI v1.0

4. Figure 4. Medical Categories view, showing symptoms organized by category.

MedAssist AI

Good afternoon, there

Search symptoms... 35x

Prasamhita@g...

MENU

- Dashboard
- Symptom Checker
- Reports
- Profile
- Settings

Let's analyze your symptoms

Tell us how you're feeling and our AI will assess possible conditions.

Step 1 of 3 33%

Search symptoms (e.g. headache, fever)...

General Cardiology Pulmonology Dermatology Gastroenterology ENT

Pain & Discomfort Mental Health Gynecology Musculoskeletal Ophthalmology

Urology

acne or pimples change in skin mole size or color itching of skin

nailbiting skin dryness, peeling, scaliness, or roughness skin growth

skin irritation skin lesion skin moles

skin oiliness skin on arm or hand looks infected skin on head or neck looks infected

skin on leg or foot looks infected skin rash skin swelling

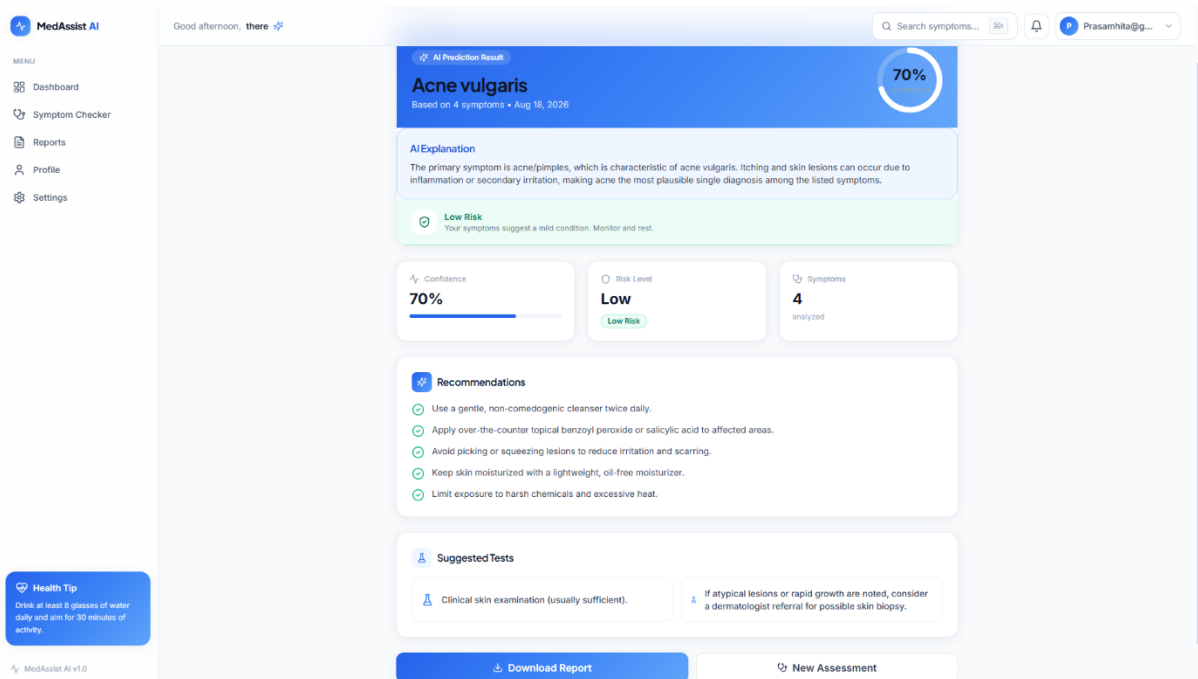
fungal lesions too little hair unwanted hair

Health Tip

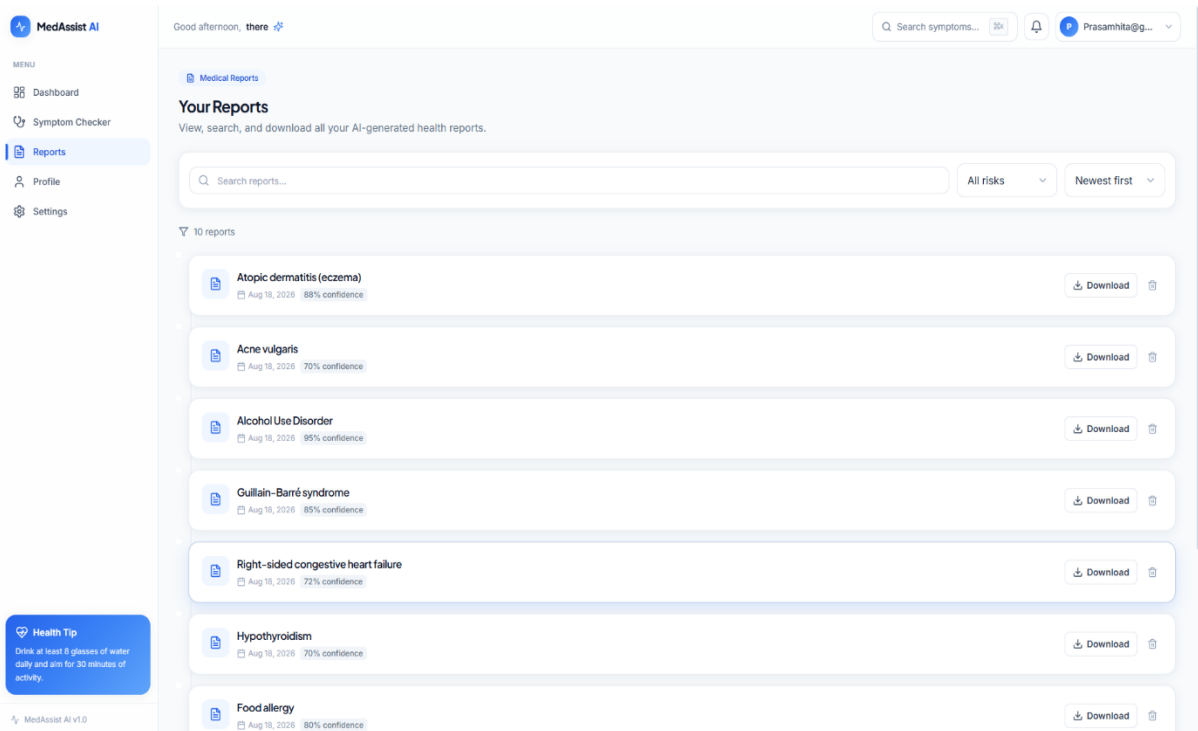
Drink at least 8 glasses of water daily and aim for 30 minutes of activity.

MedAssist AI v1.0

5. Figure 5. Prediction Result page, showing the predicted condition, confidence, risk level, and recommendations.



6. Figure 6. Reports page, showing the user's saved consultation history.



7. Figure 7. Downloaded PDF Report generated from a consultation.

## MedAssist AI

### Artificial Intelligence Medical Assessment Report

|                   |                                      |
|-------------------|--------------------------------------|
| Report ID         | bdf5f395-d957-448c-9c2c-febdebe0c498 |
| Generated On      | 18-08-2026 09:01 AM                  |
| Predicted Disease | Atopic dermatitis (eczema)           |
| Confidence        | 88 %                                 |
| Risk Level        | Low                                  |

### Symptoms Reported

- skin irritation
- skin dryness, peeling, scaliness, or roughness
- acne or pimples
- itching of skin
- skin rash

### AI Diagnosis

**Disease:** Atopic dermatitis (eczema)

**Confidence:** 88%

**Risk Level:** Low

### Recommendations

1. Apply fragrance-free moisturizers multiple times daily
2. Use mild, non-soap cleansers
3. Avoid known irritants and allergens
4. Consider low-potency topical corticosteroids for flare-ups
5. Maintain short, cool showers

### Overall Assessment

The patient is predicted to have **Atopic dermatitis (eczema)** with a confidence of **88%**. The assessed risk level is **Low**.

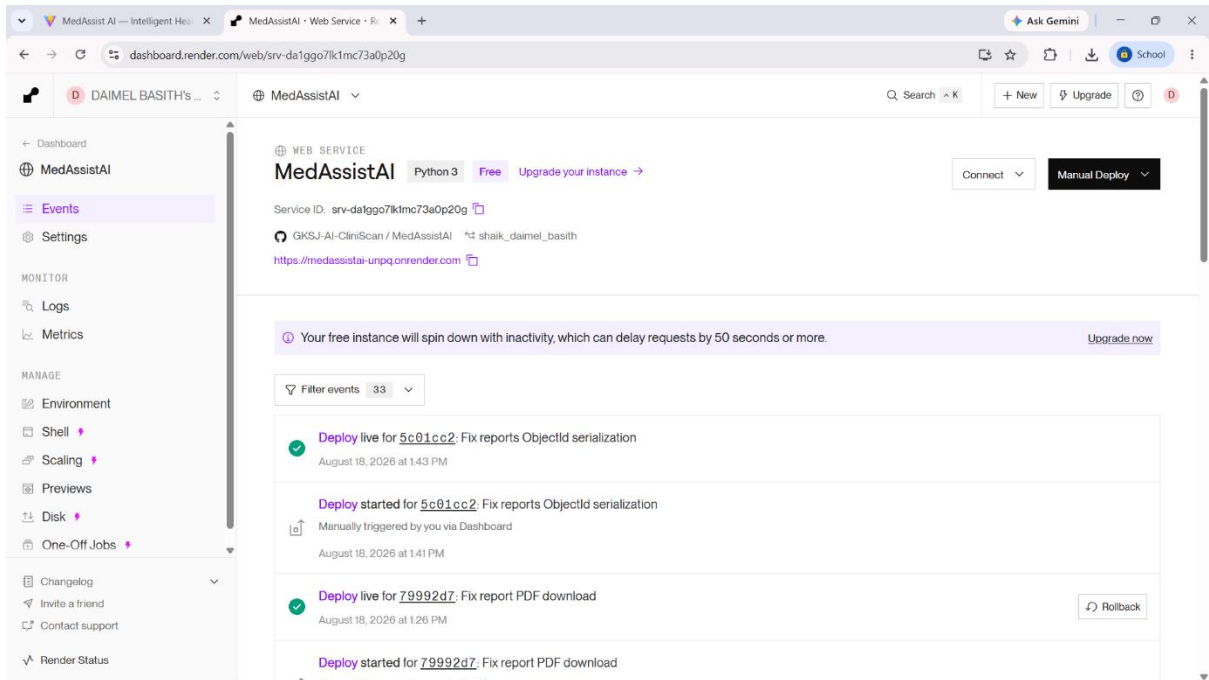
### Medical Disclaimer

This report is AI-generated and should not replace a professional medical diagnosis. Please consult a qualified healthcare professional before taking any medical decision.

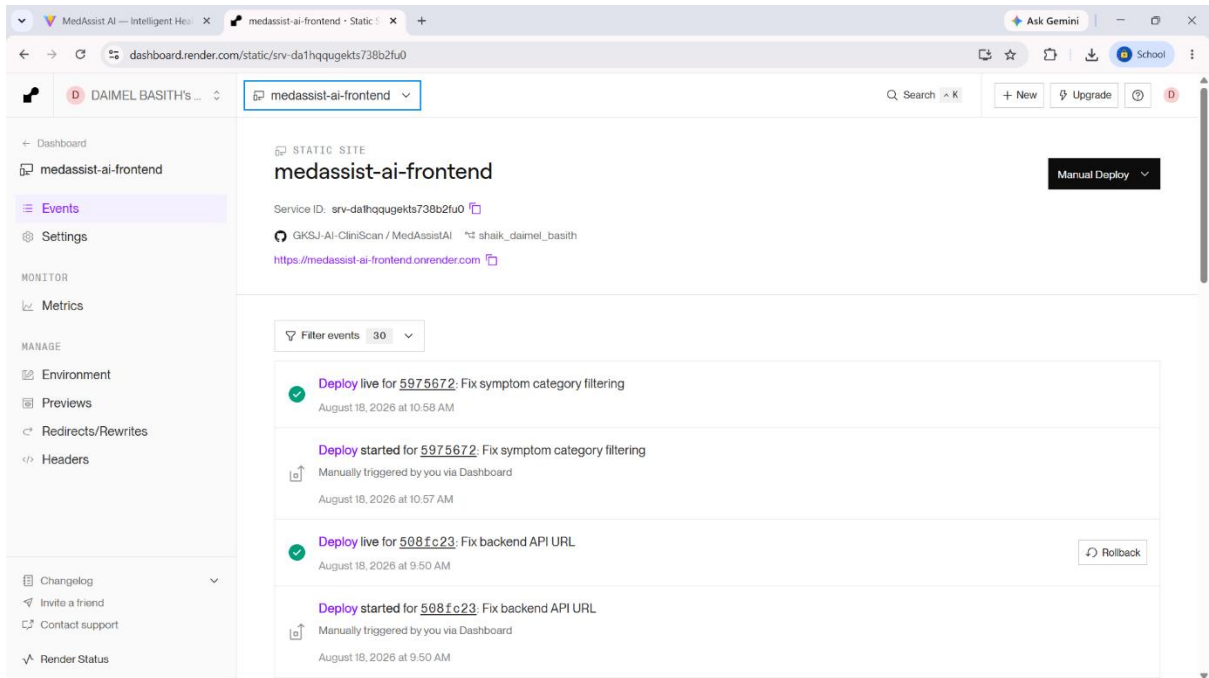
8. Figure 8. Swagger API Documentation for the MedAssist AI backend.

|                                                                             |                                                   |
|-----------------------------------------------------------------------------|---------------------------------------------------|
| <b>MedAssist AI Backend</b> <small>0.1.0</small> <small>OpenAPI 3.0</small> |                                                   |
| <a href="#">OpenAPI JSON</a>                                                |                                                   |
| <a href="#">Authorize</a>                                                   |                                                   |
| <b>Authentication</b>                                                       |                                                   |
| POST                                                                        | /api/auth/register Register                       |
| POST                                                                        | /api/auth/login Login                             |
| POST                                                                        | /api/auth/google Google                           |
| GET                                                                         | /api/auth/me Get Me                               |
| <b>Patient Profile</b>                                                      |                                                   |
| GET                                                                         | /api/profile Get Profile                          |
| PUT                                                                         | /api/profile Update Profile                       |
| <b>Symptoms Catalog</b>                                                     |                                                   |
| GET                                                                         | /api/symptoms Get Symptoms                        |
| GET                                                                         | /api/symptoms/search Search Symptoms              |
| GET                                                                         | /api/symptoms/categories Categories               |
| <b>Consultation History</b>                                                 |                                                   |
| POST                                                                        | /api/history/check Check Symptoms                 |
| GET                                                                         | /api/history Get Consultation History             |
| GET                                                                         | /api/history/{id} Get Consultation                |
| <b>Dashboard</b>                                                            |                                                   |
| GET                                                                         | /api/dashboard Dashboard                          |
| <b>Predictions</b>                                                          |                                                   |
| POST                                                                        | /api/predictions/analyze Analyze                  |
| GET                                                                         | /api/predictions/{prediction_id} Get Prediction   |
| <b>Users</b>                                                                |                                                   |
| GET                                                                         | /api/users/me Get Me                              |
| PUT                                                                         | /api/users/me Update Me                           |
| PUT                                                                         | /api/users/me/settings Update Settings            |
| <b>Reports</b>                                                              |                                                   |
| GET                                                                         | /api/reports Get Reports                          |
| GET                                                                         | /api/reports/{report_id}/download Download Report |
| DELETE                                                                      | /api/reports/{report_id} Delete Report            |
| <b>Notifications</b>                                                        |                                                   |
| GET                                                                         | /api/notifications Notifications                  |
| <b>Health</b>                                                               |                                                   |
| GET                                                                         | / Root                                            |

9. Figure 9. Render Backend Deployment dashboard, showing the deployed backend service.



10. Figure 10. Render Frontend Deployment dashboard, showing the deployed frontend service.





---

### 13. Deployment Links

| Component         | URL                                                                                                 |
|-------------------|-----------------------------------------------------------------------------------------------------|
| Frontend          | <a href="https://medassist-ai-frontend.onrender.com">https://medassist-ai-frontend.onrender.com</a> |
| Backend           | <a href="https://medassistai-unpq.onrender.com">https://medassistai-unpq.onrender.com</a>           |
| API Documentation | <a href="https://medassistai-unpq.onrender.com/docs">https://medassistai-unpq.onrender.com/docs</a> |

---

### 14. GitHub Repository Structure

MedAssistAI/

└─ backend/

└─ medassist-ai-frontend/

└─ screenshots/

└─ MILESTONE\_4\_DOCUMENTATION.md

*Adjust this structure if the actual repository layout differs (e.g., additional config or deployment files).*

---

### 15. Security Considerations

- User authentication is handled using JWT tokens, which are required to access protected API endpoints.
  - API endpoints related to user-specific data (predictions, reports) are protected and require a valid authenticated session.
  - Sensitive configuration values, including API keys and database credentials, are managed through environment variables rather than being hard-coded into the source code.
  - API keys and secrets are not committed to the GitHub repository.
  - The application is served over HTTPS in production via Render.
  - Reports and consultation records are scoped to the authenticated user, preventing access to another user's data.
-

## 16. Limitations

- AI predictions are informational only and are not a substitute for professional medical diagnosis or treatment.
  - AI-generated output may not always be medically accurate and should be interpreted with appropriate caution.
  - Users experiencing serious or emergency symptoms should seek professional medical consultation rather than relying solely on the application.
  - The AI prediction functionality depends on the availability of the Groq API and an active internet connection.
  - As the application is deployed on Render's free tier, cold-start delays may occur after periods of inactivity.
- 

## 17. Future Enhancements

The following are proposed as future work and are **not** part of the current implementation:

- Integration with doctor consultation or telemedicine services.
  - Improved medical knowledge grounding for AI predictions (e.g., integration with verified medical knowledge bases).
  - Multilingual support for a broader user base.
  - Enhanced validation mechanisms for AI-generated predictions.
  - Richer analytics and health trend visualization for users.
  - Notification system for report updates or health reminders.
  - Development of a dedicated mobile application.
  - Additional clinical safety mechanisms and disclaimers to further reinforce the informational nature of the system.
- 

## 18. Conclusion

Milestone 4 marks the completion of the core implementation, integration, testing, and deployment of the MedAssist AI application. The system successfully allows users to authenticate, select symptoms across multiple medical categories, receive AI-based informational predictions via the Groq API, maintain a history of consultations, and download reports as PDF documents. Key implementation and deployment issues

encountered during this phase were identified and resolved, resulting in a functional, deployed application accessible via the links provided in this document.

---

## 19. Demo / Deployment Summary

MedAssist AI is deployed and accessible at the following locations:

- **Live Application (Frontend):** <https://medassist-ai-frontend.onrender.com>
- **Backend API:** <https://medassistai-unpq.onrender.com>
- **API Documentation (Swagger UI):** <https://medassistai-unpq.onrender.com/docs>

The application demonstrates a complete end-to-end workflow: user authentication, category-based symptom selection, AI-based prediction using the Groq API (openai/gpt-oss-120b), persistent storage in MongoDB, and downloadable PDF medical reports.

---

## Information That Needs My Confirmation

The following items could not be verified from the material provided and should be confirmed or filled in before final submission:

1. **Dashboard contents (Section 5.2):** Specific data/widgets displayed on the dashboard (e.g., recent consultations, health summaries, quick action buttons).
2. **User Profile / additional features (Section 5.8):** Any implemented features beyond those explicitly listed (e.g., profile editing, account settings, password reset).
3. **Full API endpoint list and schemas (Section 6):** Additional endpoints, request/response body schemas, query parameters, and error responses beyond the confirmed list — recommend cross-checking against /docs (Swagger UI) or backend route files.
4. **Database field-level schema (Section 7):** Specific fields within each MongoDB collection (users, profiles, symptoms, disease\_profiles, consultations).
5. **Screenshots (Section 12):** All screenshot image files need to be provided and placed in the screenshots/ folder; captions may need adjustment once actual screenshots are reviewed.

6. **Actual GitHub repository structure (Section 14):** Confirm whether the folder structure matches the recommended layout, including any additional files (e.g., .env.example, requirements.txt, README.md, CI/CD configuration).
7. **Team/author details, submission metadata:** Course name, team member names, roll numbers, submission date, or institution details, if required by your submission format (not included in this document, as none were provided).